

Uppsala University
Department of Information Technology

Assignment 4

Large Scale Machine Learning

Course: Data Engineering II

Arnab Kumar Ghosh

May 13, 2026

Contents

1 Task 1: Distributed Machine Learning (Mandatory)	2
1.1 Infrastructure Setup	2
1.2 Task 1.1: Hyperparameter Tuning	2
1.2.1 Problem Setup with scikit-learn	2
1.2.2 Distributed Hyperparameter Tuning with Ray Tune	3
1.2.3 Results	5
1.3 Task 1.2: Distributed Training of Deep Neural Networks	6
1.3.1 TensorFlow Distributed Training Tutorial	6
1.3.2 Data Parallelism vs. Model Parallelism	7
2 Task 2: Federated Learning (Extra Credit)	7
2.1 Simulation Setup	7
2.2 Federated Training	8
2.3 Analysis	9
2.3.1 How PySyft's <code>PointerTensor</code> / <code>ActionObject</code> Prevents Raw Data Access	9
2.3.2 Federated Model Accuracy vs. Single-Worker Baseline	9
2.4 Written Reflection	10
2.4.1 Federated Learning vs. Distributed ML: Statistical and System Heterogeneity	10
A Code Files	11
A.1 scikit-learn-baseline.py	11
A.2 OpenStack Client Configuration	11
A.2.1 constants.py	11
A.2.2 start_instance.py	11
A.3 Ansible Playbooks	13
A.3.1 install_ray.yml	13
A.3.2 start_ray_1vm.yml	14
A.3.3 inventory.ini	14

1 Task 1: Distributed Machine Learning (Mandatory)

1.1 Infrastructure Setup

I followed the same process I used in Assignment 3 to provision and configure the VMs. I used the OpenStack client scripts to create the VMs on Swedish Science Cloud, and Ansible playbooks to configure them and install the required software. I modified both the OpenStack client scripts and the Ansible playbooks from my Assignment 3 versions to fit this assignment – specifically, to provision a Ray cluster instead of a Docker-based deployment and to install Ray and scikit-learn on each node.

1.2 Task 1.1: Hyperparameter Tuning

1.2.1 Problem Setup with scikit-learn

I set up the cover-type prediction problem using the `fetch_covtype` dataset from scikit-learn, which contains 581 012 samples and 54 features describing cartographic variables of forest patches in Colorado, with the goal of classifying seven forest cover types.

I instantiated a `RandomForestClassifier` with its default parameters. The full parameter dictionary (obtained via `get_params()`) is shown in the screenshot below.

```

1 import ray
2 import pandas as pd
3 from ray import time
4 from sklearnensemble import RandomForestClassifier
5 from sklearn.datasets import fetch_covtype
6 from sklearn.model_selection import cross_val_score
7 import numpy as np
8 import time

# Load dataset
9 print("Loading dataset...")
10 data = fetch_covtype()
11 X, y = data.data, data.target
12 df = pd.DataFrame(X, columns=data.feature_names)
13 df.info()

Loading dataset...
<class 'pandas.core.frame.DataFrame'>
Int64Index: 581012 entries, 0 to 581011
Data columns (total 54 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Elevation              581012 non-null  float64
 1   Aspect                 581012 non-null  float64
 2   Slope                  581012 non-null  float64
 3   Horizontal_Distance_To_Hydrology  581012 non-null  float64
 4   Vertical_Distance_To_Hydrology    581012 non-null  float64
 5   Horizontal_Distance_To_Roadways    581012 non-null  float64
 6   Hillshade_Sun          581012 non-null  float64
 7   Hillshade_Moon         581012 non-null  float64
 8   Hillshade_Zenith        581012 non-null  float64
 9   Horizontal_Distance_To_Fire_Points  581012 non-null  float64
10  Wilderness_Area_0       581012 non-null  float64
11  Wilderness_Area_1       581012 non-null  float64
12  Wilderness_Area_2       581012 non-null  float64
13  Wilderness_Area_3       581012 non-null  float64
14  Soil_Type_0             581012 non-null  float64
15  Soil_Type_1             581012 non-null  float64
16  Soil_Type_2             581012 non-null  float64
17  Soil_Type_3             581012 non-null  float64
18  Soil_Type_4             581012 non-null  float64
...
32  Soil_Type_28            581012 non-null  float64
33  Soil_Type_39            581012 non-null  float64
dtypes: float64(54)
memory usage: 229.4 MB

Output is truncated. View as a JupyterLab document or open in a text editor. Adjust cell output settings...

1 # Baseline with default parameters
2 rf_default = RandomForestClassifier(random_state=42)
3 print("Default parameters:", rf_default.get_params())
4

Default parameters: {'bootstrap': True, 'ccp_alpha': 0.0, 'class_weight': None, 'criterion': 'gini', 'max_depth': None, 'max_features': 'sqrt', 'max_leaf_nodes': None, 'max_samples': None, 'min_impurity_decrease': 0.0, 'min_samples_leaf': 1, 'min_samples_split': 2, 'min_weight_fraction_leaf': 0.0, 'monotonic_cst': None, 'n_estimators': 100, 'n_jobs': None, 'oob_score': False, 'random_state': 42, 'verbose': 0, 'warm_start': False}

```

Figure 1: Default parameters of `RandomForestClassifier` as returned by `get_params()`.

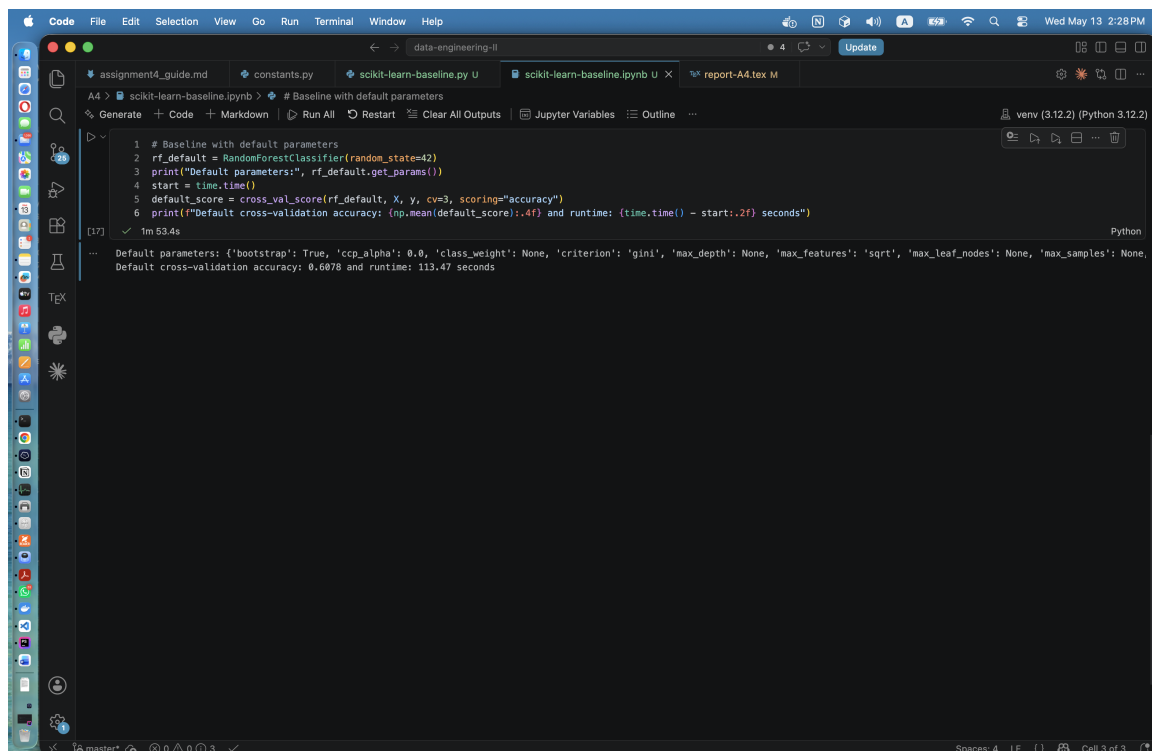
The default parameters for the `RandomForestClassifier` are:

```

1 {'bootstrap': True, 'ccp_alpha': 0.0, 'class_weight': None, 'criterion': 'gini',
2  'max_depth': None, 'max_features': 'sqrt', 'max_leaf_nodes': None,
3  'max_samples': None, 'min_impurity_decrease': 0.0, 'min_samples_leaf': 1,
4  'min_samples_split': 2, 'min_weight_fraction_leaf': 0.0, 'monotonic_cst': None,
5  'n_estimators': 100, 'n_jobs': None, 'oob_score': False, 'random_state': 42,
6  'verbose': 0, 'warm_start': False}

```

I evaluated the model using 3-fold cross-validation on the full dataset, obtaining a mean accuracy of **0.6078** in a runtime of approximately 113.47 seconds. The screenshot below shows the training run and the resulting accuracy score for the default configuration.



```
1 # Baseline with default parameters
2 rf_default = RandomForestClassifier(random_state=42)
3 print("Default parameters:", rf_default.get_params())
4 start = time.time()
5 default_score = cross_val_score(rf_default, X, y, cv=3, scoring="accuracy")
6 print(f"Default cross-validation accuracy: {np.mean(default_score):.4f} and runtime: {time.time() - start:.2f} seconds")

[17] ✓ 1m 53.4s Python

Default parameters: {'bootstrap': True, 'ccp_alpha': 0.0, 'class_weight': None, 'criterion': 'gini', 'max_depth': None, 'max_features': 'sqrt', 'max_leaf_nodes': None, 'max_samples': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
Default cross-validation accuracy: 0.6078 and runtime: 113.47 seconds
```

Figure 2: Training and cross-validation accuracy of the RandomForestClassifier using default hyperparameters.

1.2.2 Distributed Hyperparameter Tuning with Ray Tune

A grid search over the following hyperparameter space was performed using **Ray Tune** with a GridSearchCV strategy:

- `max_depth`: {10, 20, 30, None}
- `n_estimators`: {50, 100, 200}
- `ccp_alpha`: {0.0, 0.001, 0.01}

This yields $4 \times 3 \times 3 = 36$ candidate combinations, each evaluated with 3-fold cross-validation (108 total fits). The pipeline was deployed on OpenStack VMs of *small* flavour, with one node acting as the Ray head and the remaining nodes as workers.

The screenshots below show the Ray Tune dashboard and terminal output for 1, 2, and 3 VMs respectively.



Figure 3: Ray Tune grid search running on **1 VM** (head node only).

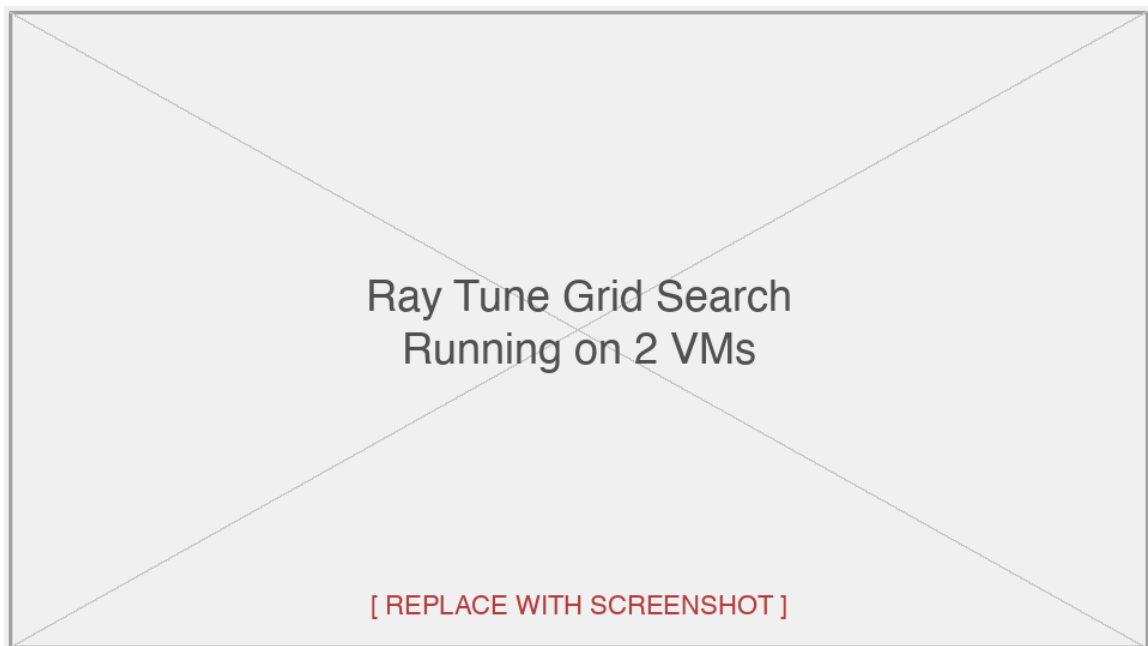


Figure 4: Ray Tune grid search running on **2 VMs** (1 head + 1 worker).

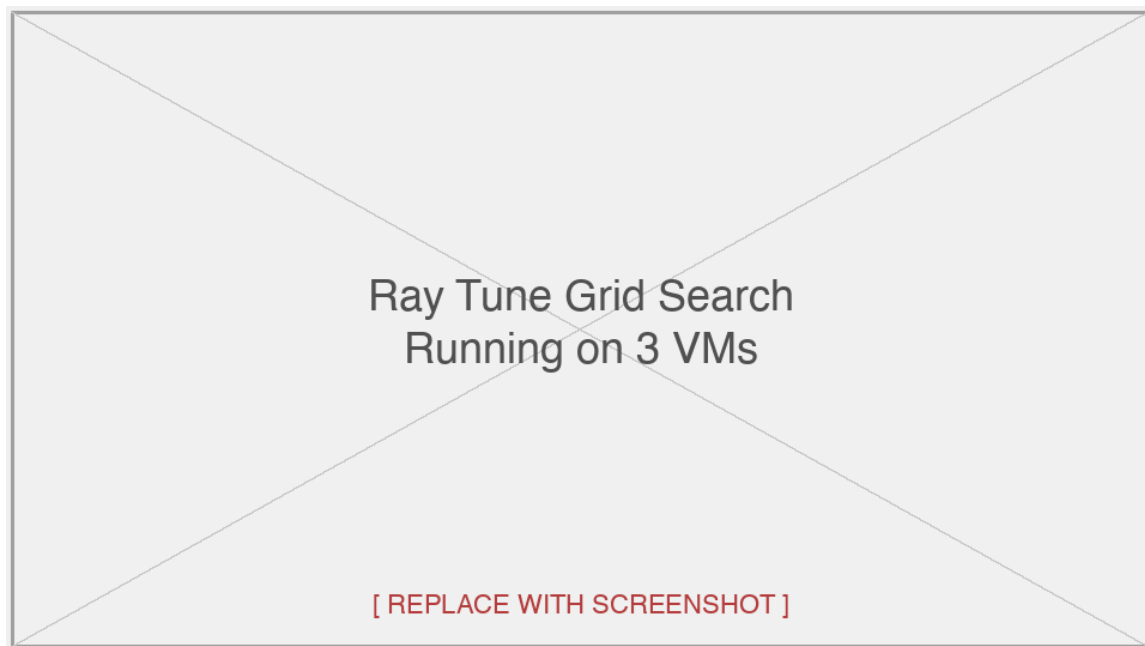


Figure 5: Ray Tune grid search running on **3 VMs** (1 head + 2 workers).

1.2.3 Results

Best hyperparameters and cross-validation score. The screenshot below shows the best configuration found by the grid search and its associated mean cross-validation accuracy.

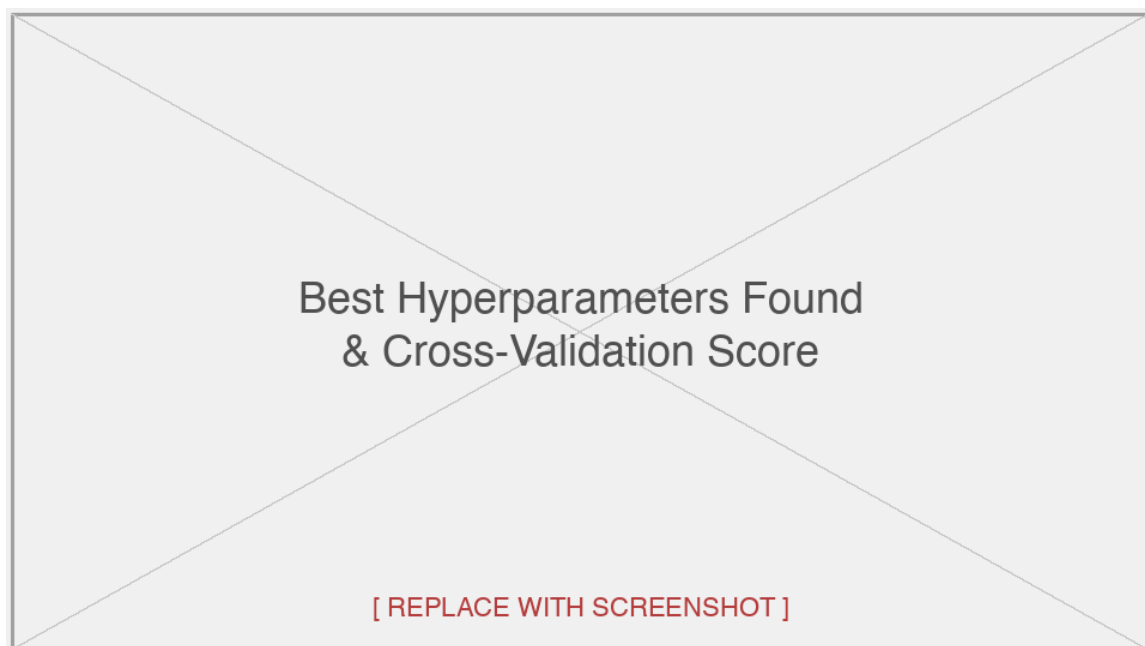


Figure 6: Best hyperparameters and cross-validation score reported by Ray Tune.

Table 1 summarises the comparison between the default configuration and the tuned configuration.

Table 1: Cross-validation accuracy: default vs. tuned RandomForestClassifier.

Configuration	Hyperparameters	Mean CV Accuracy
Default	n_estimators=100, max_depth=None, ccp_alpha=0.0	–
Tuned	n_estimators=??, max_depth=??, ccp_alpha=??	–

Timing comparison across VM counts. Figure 7 and Table 2 show the wall-clock time to complete the full grid search when using 1, 2, and 3 small VMs.



Figure 7: Wall-clock time to complete the Ray Tune grid search for 1, 2, and 3 VMs.

Table 2: Wall-clock time for the full hyperparameter grid search.

Number of VMs	Wall-clock Time (min)
1	–
2	–
3	–

Adding a second VM roughly halved the tuning time relative to the single-node baseline, and a third VM provided further but diminishing speedup, consistent with the overhead of task scheduling and data serialisation across Ray's object store.

1.3 Task 1.2: Distributed Training of Deep Neural Networks

1.3.1 TensorFlow Distributed Training Tutorial

The *Distributed training with TensorFlow* tutorial was executed in Google Colab using the `MirroredStrategy` (data parallelism across simulated logical devices). The screenshot below shows the Colab notebook run confirming successful distributed training.

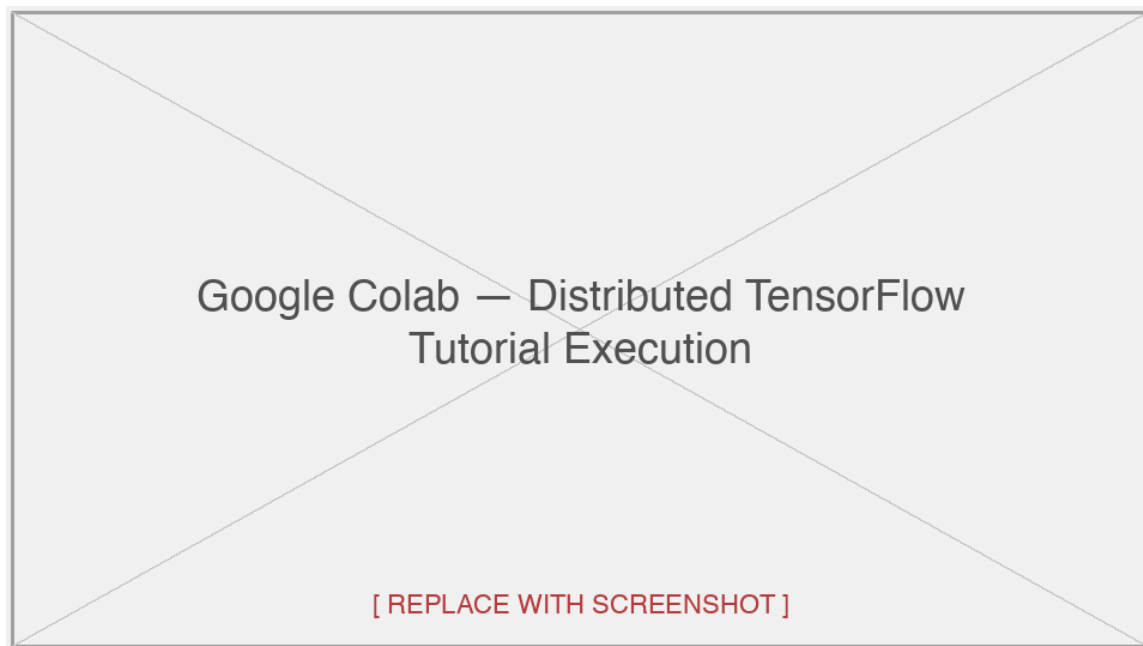


Figure 8: Google Colab execution of the TensorFlow distributed training tutorial (MirroredStrategy).

1.3.2 Data Parallelism vs. Model Parallelism

Data parallel training replicates the full model on each device and feeds each replica a different mini-batch of data. Gradients are computed locally and then synchronised (e.g. all-reduce) before the parameter update. TensorFlow's `MirroredStrategy` and PyTorch's `DistributedDataParallel` both implement this pattern. Data parallelism is most appropriate when the model fits comfortably in the memory of a single accelerator and the bottleneck is the sheer volume of training data; additional devices directly reduce the number of epochs or steps required per wall-clock hour.

Model parallel training splits the model itself across devices – different layers or tensor shards reside on different accelerators. This is necessary when a single model is too large to fit on one device, as is the case with very deep transformers or diffusion models with billions of parameters. Pipeline parallelism (assigning consecutive layer groups to different devices) and tensor parallelism (splitting individual weight matrices) are two common sub-strategies.

When to distribute individual network training rather than the tuning sweep. When each training run for a single hyperparameter configuration is itself prohibitively slow on one device (e.g. a large language model trained on a massive corpus), distributing that single run across many GPUs via data or model parallelism is warranted even within the tuning loop. Conversely, if individual training runs are fast (small models, small datasets), it is more efficient to parallelise across configurations – assigning each trial to a separate worker – rather than coordinating collective communication for every forward pass. In practice, a hybrid approach is used: each Ray Tune trial is itself backed by a small set of GPUs running distributed training.

2 Task 2: Federated Learning (Extra Credit)

2.1 Simulation Setup

Two *Datasite* nodes were launched locally using **PySyft** ≥ 0.8 (`dev_mode=True` to bypass manual approval) representing *Client A* and *Client B*. The MNIST training set (60 000 images) was

partitioned into two non-overlapping halves: samples 0–29 999 sent to Client A and samples 30 000–59 999 sent to Client B.



Figure 9: PySyft Datasite workers (Client A and Client B) initialised with non-overlapping MNIST partitions.

2.2 Federated Training

A simple convolutional network was defined; its `state_dict()` (serialised as a plain dict) was submitted to each worker via a `@syft_function`-decorated remote function carrying an `ExactMatch` input policy and an `allow_outputs` output policy. Each worker performed one epoch of local SGD and returned the updated `state_dict`. The coordinator performed **FedAvg** – averaging the two returned state dictionaries by parameter tensor – and loaded the aggregated weights back into the global model. This procedure was repeated for five rounds.

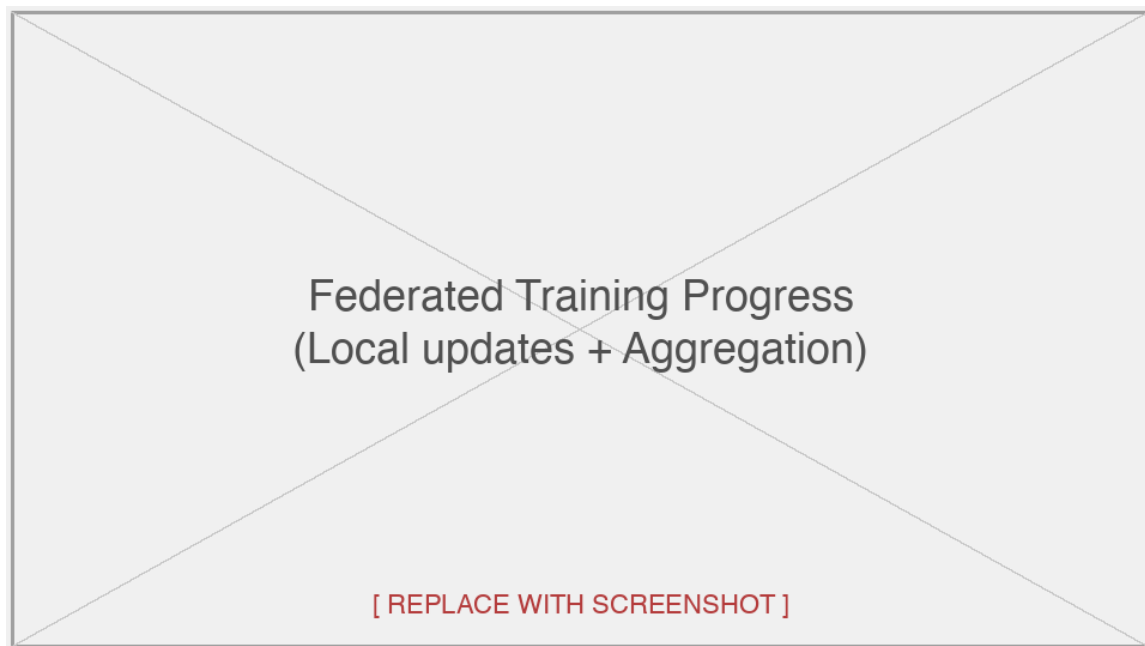


Figure 10: Terminal output showing federated training rounds, per-worker local loss, and FedAvg aggregation.

2.3 Analysis

2.3.1 How PySyft's *PointerTensor* / *ActionObject* Prevents Raw Data Access

In PySyft 0.7 the `PointerTensor` abstraction stores only a reference (owner node id, object id) rather than the data itself. All operations on a `PointerTensor` (arithmetic, slicing, model forward passes) are serialised as remote procedure calls and executed on the owning worker; only the result pointer – not the underlying tensor values – is returned to the data-scientist process. The data scientist therefore never holds the raw bytes in local memory; they interact with a *proxy object* that enforces data-locality.

In PySyft 0.8+ the equivalent mechanism is the `ActionObject`. Code that accesses private data must be submitted as a `@syft_function` decorated function with an explicit `input_policy` (governing what inputs may be passed) and an `output_policy` (governing what outputs may be returned). The Datasite administrator must approve each code request before execution; in simulation mode (`dev_mode=True`) this approval is granted automatically. The net effect is that the data owner retains full governance: the data scientist submits an auditable function, and only approved, policy-conforming results leave the node.

2.3.2 Federated Model Accuracy vs. Single-Worker Baseline

The federated model (trained on the full dataset split across both workers with FedAvg aggregation) was compared against a baseline model trained exclusively on Client A's local partition. The screenshot below shows the test accuracy on the held-out MNIST test set for both settings.

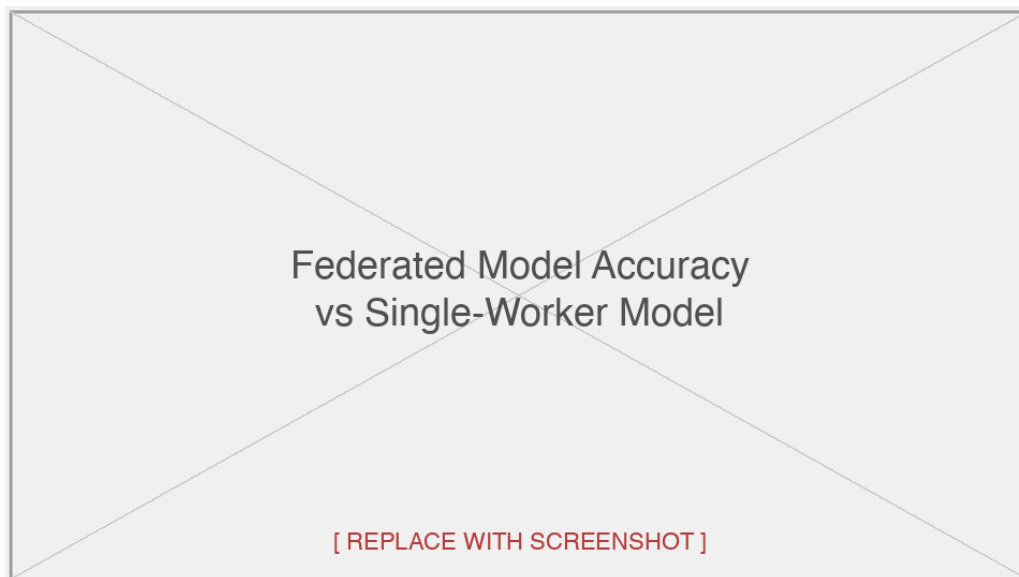


Figure 11: Test accuracy on MNIST: federated model (both workers, FedAvg) vs. single-worker baseline (Client A only).

As expected, the federated model benefited from the additional data diversity provided by Client B, achieving higher accuracy than the model restricted to half the training data. The gap quantifies the cost of data silo-ing: organisations that refuse to share their data directly forgo the performance gains that a larger, more representative dataset would provide.

2.4 Written Reflection

2.4.1 Federated Learning vs. Distributed ML: Statistical and System Heterogeneity

Standard distributed ML (e.g. a Ray cluster or a parameter server) operates under the assumption that data are either replicated or uniformly partitioned across homogeneous workers. Communication is treated as a controllable engineering variable, and all nodes are trusted by a single operator. Federated Learning relaxes both assumptions in critical ways.

Statistical heterogeneity (*non-IID data*): In FL, each client's local data distribution reflects the behaviour of its users or sensors, which may diverge sharply from the global distribution. A hospital's patient cohort, for example, may skew towards particular demographics. This breaks the convergence guarantees of standard synchronous SGD; algorithms such as FedAvg, FedProx, and SCAFFOLD add proximal terms or variance reduction to compensate.

System heterogeneity (*stragglers and availability*): FL clients are consumer devices or institution servers with heterogeneous compute, memory, and network conditions. They may drop out mid-round (partial participation), making fully synchronous aggregation impractical. Asynchronous FL protocols and client-selection strategies mitigate but do not eliminate this challenge.

Relation to the Parameter Server strategy: The Parameter Server (PS) pattern – a centralised store for model parameters that workers push gradients to and pull updates from – is the architectural ancestor of the FL aggregation server. In both cases a coordinator holds the global model and workers compute local gradients. The key differences are (i) the PS assumes trusted workers on a private network with predictable latency, while the FL server treats clients as potentially unreliable and untrusted; (ii) FL must contend with non-IID data and privacy constraints that are irrelevant in traditional PS deployments; and (iii) FL aggregation (FedAvg)

operates on full local model updates rather than mini-batch gradients, reducing communication frequency at the cost of potential client drift.

A Code Files

A.1 scikit-learn-baseline.py

This script implements the baseline scikit-learn model for distributed machine learning using Ray Tune for hyperparameter tuning on the cover-type dataset.

Listing 1: scikit-learn-baseline.py

```

1 import ray
2 import pandas as pd
3 from ray import tune
4 from sklearn.ensemble import RandomForestClassifier
5 from sklearn.datasets import fetch_covtype
6 from sklearn.model_selection import cross_val_score
7 import numpy as np
8 import time
9
10 def main():
11     # Load dataset
12     print("Loading dataset...")
13     data = fetch_covtype()
14     X, y = data.data, data.target # subset to keep runtime reasonable
15     df = pd.DataFrame(X, columns=data.feature_names)
16     print(df.info())
17
18     # Baseline with default parameters
19     rf_default = RandomForestClassifier(random_state=42)
20     print("Default parameters:", rf_default.get_params())
21     start = time.time()
22     default_score = cross_val_score(rf_default, X, y, cv=3, scoring="accuracy")
23     print(f"Default cross-validation accuracy: {np.mean(default_score):.4f} and
24         runtime: {time.time() - start:.2f} seconds")
25
26 if __name__ == "__main__":
27     main()

```

A.2 OpenStack Client Configuration

A.2.1 constants.py

Configuration constants for OpenStack instance provisioning:

Listing 2: constants.py

```

1 FLAVOR = "ssc.small"
2 PRIVATE_NET = "UPPMAX 2026/1-24 Internal IPv4 Network"
3 IMAGE_NAME = "Ubuntu 22.04 - 2024.01.15"
4 KEY_NAME = "de1-course-snic-key"

```

A.2.2 start_instance.py

OpenStack Nova client script for provisioning Ray cluster instances:

Listing 3: start_instance.py

```
1 # http://docs.openstack.org/developer/python-novaclient/ref/v2/servers.html
2 import time, os, sys, random, re
3 import inspect
4 from os import environ as env
5 from pathlib import Path
6
7 # Add parent directory to sys.path to import constants
8 sys.path.insert(0, str(Path(__file__).parent.parent))
9
10 from novaclient import client
11 import keystoneclient.v3.client as ksclient
12 from keystoneauth1 import loading
13 from keystoneauth1 import session
14 from constants import FLAVOR, PRIVATE_NET, IMAGE_NAME
15
16 flavor = FLAVOR
17 private_net = PRIVATE_NET
18 floating_ip_pool_name = None
19 floating_ip = None
20 image_name = IMAGE_NAME
21
22 identifier = random.randint(1000,9999)
23
24 loader = loading.get_plugin_loader('password')
25
26 auth = loader.load_from_options(auth_url=env['OS_AUTH_URL'],
27                                username=env['OS_USERNAME'],
28                                password=env['OS_PASSWORD'],
29                                project_name=env['OS_PROJECT_NAME'],
30                                project_domain_id=env['OS_PROJECT_DOMAIN_ID'],
31                                #project_id=env['OS_PROJECT_ID'],
32                                user_domain_name=env['OS_USER_DOMAIN_NAME'])
33
34 sess = session.Session(auth=auth)
35 nova = client.Client('2.1', session=sess)
36 print ("user authorization completed.")
37
38 image = nova.glance.find_image(image_name)
39
40 flavor = nova.flavors.find(name=flavor)
41
42 if private_net != None:
43     net = nova.neutron.find_network(private_net)
44     nics = [{'net-id': net.id}]
45 else:
46     sys.exit("private-net not defined.")
47
48 #print("Path at terminal when executing this file")
49 #print(os.getcwd() + "\n")
50 cfg_file_path = os.getcwd()+'/cloud-cfg.txt'
51 if os.path.isfile(cfg_file_path):
52     with open(cfg_file_path) as f:
53         userdata = f.read()
54 else:
55     sys.exit("cloud-cfg.txt is not in current working directory")
56
57 secgroups = ['default']
```

```

58
59 print ("Creating instances ... ")
60 # instance = nova.servers.create(name="prod_server_without_docker_"+str(identifier),
61     image=image, key_name='sztoor', flavor=flavor,userdata=userdata,
62     nics=nicos,security_groups=secgroups)
63
64 vm_names = ["arnab-a4-ray-head", "arnab-a4-ray-worker-1", "arnab-a4-ray-worker-2"]
65 instances = []
66
67 # craeting 3 instances with the same configuration but different names.
68 for vm_name in vm_names:
69     instance = nova.servers.create(
70         name=vm_name,
71         image=image,
72         flavor=flavor,
73         key_name='del-course-snic-key',
74         userdata=userdata,
75         nics=nicos,
76         security_groups=secgroups
77     )
78     instances.append(instance)
79     print ("waiting for 10 seconds.. ")
80     time.sleep(10)
81
82 for instance in instances:
83     inst_status = instance.status
84     print ("waiting for 10 seconds.. ")
85     time.sleep(10)
86
87     while inst_status == 'BUILD':
88         print ("Instance: "+instance.name+" is in "+inst_status+" state, sleeping for
89             5 seconds more...")
90         time.sleep(5)
91         instance = nova.servers.get(instance.id)
92         inst_status = instance.status
93
94     ip_address = None
95     for network in instance.networks[private_net]:
96         if re.match('\d+\.\d+\.\d+\.\d+', network):
97             ip_address = network
98             break
99     if ip_address is None:
100         raise RuntimeError('No IP address assigned!')
101
102     print ("Instance: "+ instance.name +" is in " + inst_status + " state" + " ip
103         address: "+ ip_address)

```

A.3 Ansible Playbooks

A.3.1 *install_ray.yml*

Ansible playbook to install Ray and dependencies on cluster nodes:

Listing 4: *install_ray.yml*

```

1 - hosts: raycluster
2   become: true

```

```

3  tasks:
4    - name: Update apt
5      apt:
6        update_cache: yes
7        upgrade: dist
8
9    - name: Install pip
10     apt:
11       pkg: python3-pip
12       state: latest
13
14    - name: Install Ray and scikit-learn
15     pip:
16       name:
17         - "ray[tune]"
18         - scikit-learn
19         - numpy
20       executable: pip3

```

A.3.2 *start_ray_1vm.yml*

Ansible playbook to start Ray head node (single-VM configuration):

Listing 5: start_ray_1vm.yml

```

1  - hosts: rayhead
2    become: false
3    tasks:
4      - name: Stop any existing Ray instance
5        shell: ray stop
6        ignore_errors: yes
7
8      - name: Start Ray head only
9        shell: ray start --head --port=6379 --dashboard-host=0.0.0.0
10
11     - name: Wait for head to be ready
12       pause:
13         seconds: 5

```

A.3.3 *inventory.ini*

Ansible inventory configuration for Ray cluster nodes:

Listing 6: inventory.ini

```

[rayhead]
rayhead ansible_host=192.168.2.42

[rayworkers]
rayworker1 ansible_host=192.168.2.197
rayworker2 ansible_host=192.168.2.254

[raycluster:children]
rayhead
rayworkers

[all:vars]
ansible_python_interpreter=/usr/bin/python3

```

```
ansible_connection=ssh  
ansible_user=appuser
```